

1. Crear una tabla pequeña de Credit Age en M0

No iría a **trades** raw. Utilizaría nuestra tabla point-in-time ya filtrada:

bu_frd.model_trades_behavioral_final_13m_2022_2025

y exclusivamente el snapshot de observation date.

```
sdf_sql(  
  sc,  
  "  
  CREATE TABLE bu_frd.eda_credit_age_2022_2025  
  STORED AS PARQUET  
  AS  
  
  WITH cohort AS (  
  
    SELECT  
      party_id,  
      observation_date,  
      observation_year,  
      label_3m  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  ),
```

credit_dates AS (

```
  SELECT  
    f.party_id,  
    f.observation_date,  
    f.observation_year,  
    f.label_3m,  
  
    MIN(  
      CASE  
        WHEN t.frst_rpt_dte IS NOT NULL  
        AND t.frst_rpt_dte <= f.observation_date  
        THEN t.frst_rpt_dte  
      END  
    ) AS earliest_frst_rpt_dte,  
  
    MIN(  
      CASE  
        WHEN t.open_dte IS NOT NULL  
        AND t.open_dte <= f.observation_date
```

```

        THEN t.open_dte
      END
    ) AS earliest_open_dte

FROM cohort f

LEFT JOIN bu_frd.model_trades_behavioral_final_13m_2022_2025 t

  ON f.party_id = t.party_id
  AND f.observation_date = t.observation_date
  AND t.data_date = f.observation_date

GROUP BY
  f.party_id,
  f.observation_date,
  f.observation_year,
  f.label_3m
)

SELECT
  *,

  CASE
    WHEN earliest_frst_rpt_dte IS NOT NULL
    THEN CAST(
      FLOOR(
        MONTHS_BETWEEN(
          observation_date,
          earliest_frst_rpt_dte
        )
      ) AS INT
    )
  END AS credit_age_months,

  CASE
    WHEN earliest_open_dte IS NOT NULL
    THEN CAST(
      FLOOR(
        MONTHS_BETWEEN(
          observation_date,
          earliest_open_dte
        )
      ) AS INT
    )
  END AS oldest_trade_age_months

FROM credit_dates

```

)

Aquí obtenemos dos medidas:

$$CreditAge = ObservationDate - EarliestFirstReportedDate$$

y, como sensitivity:

$$OldestTradeAge = ObservationDate - EarliestOpenDate$$

2. QA de integridad

Primero debemos comprobar que seguimos teniendo exactamente nuestra cohorte:

```
credit_age_integrity_qa <- sdf_sql(
  sc,
  "
  SELECT
    COUNT(*) AS total_rows,

    COUNT(DISTINCT party_id)
    AS distinct_parties,

    SUM(
      CASE WHEN label_3m = 1
        THEN 1 ELSE 0 END
    ) AS positives,

    SUM(
      CASE WHEN label_3m = 0
        THEN 1 ELSE 0 END
    ) AS negatives

  FROM bu_frd.eda_credit_age_2022_2025
  "
) %>%
  collect()
```

credit_age_integrity_qa

Esperamos:

total_rows	688589
distinct_parties	688589
positives	62599

3. Coverage de Credit Age por label

```
credit_age_coverage <- sdf_sql(  
  sc,  
  "  
  SELECT  
    label_3m,  
  
    COUNT(*) AS parties,  
  
    COUNT(credit_age_months)  
    AS parties_with_credit_age,  
  
    SUM(  
      CASE WHEN credit_age_months IS NULL  
        THEN 1 ELSE 0 END  
    ) AS missing_credit_age,  
  
    COUNT(credit_age_months) * 1.0 /  
    COUNT(*) AS credit_age_coverage,  
  
    COUNT(oldest_trade_age_months)  
    AS parties_with_oldest_trade_age  
  
  FROM bu_frd.eda_credit_age_2022_2025  
  
  GROUP BY label_3m  
  
  ORDER BY label_3m  
  "  
) %>%  
  collect()
```

credit_age_coverage

Queremos saber especialmente si la disponibilidad es similar entre:

label = 1

y

label = 0

como acabamos de hacer con `birth_dte`.

4. QA de valores de Credit Age

```
credit_age_quality <- sdf_sql(  
  sc,  
  "  
  SELECT  
    COUNT(*) AS total_parties,  
  
    SUM(  
      CASE WHEN credit_age_months IS NULL  
        THEN 1 ELSE 0 END  
    ) AS missing_credit_age,  
  
    SUM(  
      CASE WHEN credit_age_months < 0  
        THEN 1 ELSE 0 END  
    ) AS negative_credit_age,  
  
    SUM(  
      CASE WHEN credit_age_months < 12  
        THEN 1 ELSE 0 END  
    ) AS credit_age_under_12m,  
  
    MIN(credit_age_months)  
      AS min_credit_age_months,  
  
    PERCENTILE_APPROX(  
      credit_age_months, 0.01  
    ) AS p01,  
  
    PERCENTILE_APPROX(  
      credit_age_months, 0.05  
    ) AS p05,  
  
    PERCENTILE_APPROX(  
      credit_age_months, 0.25  
    ) AS p25,  
  
    PERCENTILE_APPROX(  
      credit_age_months, 0.50  
    ) AS median,  
  
    PERCENTILE_APPROX(  
      credit_age_months, 0.75
```

```

) AS p75,

PERCENTILE_APPROX(
  credit_age_months, 0.95
) AS p95,

PERCENTILE_APPROX(
  credit_age_months, 0.99
) AS p99,

MAX(credit_age_months)
  AS max_credit_age_months

FROM bu_frd.eda_credit_age_2022_2025
"
) %>%
  collect()

credit_age_quality

```

Un resultado importante será cuántos tienen:

CreditAge < 12

porque nuestro behavioral lookback necesita 12 feature months.

Eso no significa automáticamente que sean inválidos, pero tenemos que conocerlo.

5. Distribución Positive vs Negative

```

credit_age_summary_by_label <- sdf_sql(
  sc,
  "
SELECT
  label_3m,

  COUNT(*) AS parties,

  COUNT(credit_age_months)
    AS parties_with_credit_age,

  AVG(credit_age_months)
    AS avg_credit_age_months,

  STDDEV_SAMP(credit_age_months)

```

```

        AS sd_credit_age_months,

        PERCENTILE_APPROX(
            credit_age_months, 0.25
        ) AS p25,

        PERCENTILE_APPROX(
            credit_age_months, 0.50
        ) AS median,

        PERCENTILE_APPROX(
            credit_age_months, 0.75
        ) AS p75,

        PERCENTILE_APPROX(
            credit_age_months, 0.95
        ) AS p95

FROM bu_frd.eda_credit_age_2022_2025

WHERE credit_age_months IS NOT NULL

GROUP BY label_3m

ORDER BY label_3m
"
) %>%
collect()

```

credit_age_summary_by_label

Aquí veremos, por ejemplo, si:

Positive median = 150 months

Negative median = 148 months

o si existe una diferencia material.

6. SMD de Credit Age

Al igual que con edad:

$$SMD = \frac{CreditAge_{positive} - CreditAge_{negative}}{\sqrt{\frac{s_{positive}^2 + s_{negative}^2}{2}}}$$

```

credit_age_smd <- sdf_sql(
  sc,
  "
  WITH stats AS (

    SELECT
      label_3m,

      AVG(credit_age_months)
      AS mean_credit_age,

      VAR_SAMP(credit_age_months)
      AS var_credit_age

    FROM bu_frd.eda_credit_age_2022_2025

    WHERE credit_age_months IS NOT NULL

    GROUP BY label_3m
  ),

  p AS (

    SELECT
      mean_credit_age AS mean_positive,
      var_credit_age AS var_positive

    FROM stats

    WHERE label_3m = 1
  ),

  n AS (

    SELECT
      mean_credit_age AS mean_negative,
      var_credit_age AS var_negative

    FROM stats

    WHERE label_3m = 0
  )

  SELECT
    mean_positive,

```

```

mean_negative,

mean_positive - mean_negative
  AS mean_difference_months,

SQRT(
  (var_positive + var_negative) / 2.0
) AS pooled_sd,

(
  mean_positive - mean_negative
)
/
SQRT(
  (var_positive + var_negative) / 2.0
) AS standardized_mean_difference

FROM p
CROSS JOIN n
"
) %>%
collect()

credit_age_smd

```

7. Credit Age Bands

Para presentación utilizaría bandas intuitivas:

```

<12 months
12–23
24–35
36–59
60–119
120–179
180–239
240+
MISSING

```

Así podemos mostrar explícitamente el ejemplo que te pidieron de **month 24**.

```

credit_age_band_balance <- sdf_sql(
  sc,
  "
  WITH x AS (

```

```

SELECT
    party_id,
    label_3m,

CASE
    WHEN credit_age_months IS NULL
        THEN 'MISSING'

    WHEN credit_age_months < 12
        THEN '<12'

    WHEN credit_age_months BETWEEN 12 AND 23
        THEN '12-23'

    WHEN credit_age_months BETWEEN 24 AND 35
        THEN '24-35'

    WHEN credit_age_months BETWEEN 36 AND 59
        THEN '36-59'

    WHEN credit_age_months BETWEEN 60 AND 119
        THEN '60-119'

    WHEN credit_age_months BETWEEN 120 AND 179
        THEN '120-179'

    WHEN credit_age_months BETWEEN 180 AND 239
        THEN '180-239'

    ELSE '240+'
END AS credit_age_band

FROM bu_frd.eda_credit_age_2022_2025
),

counts AS (

SELECT
    credit_age_band,

SUM(
    CASE WHEN label_3m = 1
        THEN 1 ELSE 0 END
) AS positive_parties,

SUM(
    CASE WHEN label_3m = 0
        THEN 1 ELSE 0 END

```

```

        ) AS negative_parties,

        COUNT(*) AS total_parties

FROM x

GROUP BY credit_age_band
),

totals AS (

    SELECT
        SUM(positive_parties)
        AS total_positive,

        SUM(negative_parties)
        AS total_negative

    FROM counts
)

SELECT
    c.credit_age_band,

    c.positive_parties,
    c.negative_parties,
    c.total_parties,

    c.positive_parties * 1.0 /
    t.total_positive
    AS pct_positive_distribution,

    c.negative_parties * 1.0 /
    t.total_negative
    AS pct_negative_distribution,

    CASE
        WHEN c.negative_parties > 0
        THEN
            (c.positive_parties * 1.0 / t.total_positive)
            /
            (c.negative_parties * 1.0 / t.total_negative)
        END AS representation_ratio

FROM counts c

CROSS JOIN totals t

```

```

ORDER BY
  CASE c.credit_age_band
    WHEN '<12' THEN 1
    WHEN '12-23' THEN 2
    WHEN '24-35' THEN 3
    WHEN '36-59' THEN 4
    WHEN '60-119' THEN 5
    WHEN '120-179' THEN 6
    WHEN '180-239' THEN 7
    WHEN '240+' THEN 8
    ELSE 9
  END
"
) %>%
collect()

credit_age_band_balance

```

8. Esta es la validación más importante: exact credit-age support

Ahora hacemos exactamente lo que te solicitaron:

if positives at month 24 of history, then also negatives at that same age.

Primero a nivel de **mes exacto de credit age**:

```

credit_age_exact_match_validation <- sdf_sql(
  sc,
  "
  WITH x AS (

    SELECT
      observation_date,
      credit_age_months,

      SUM(
        CASE WHEN label_3m = 1
          THEN 1 ELSE 0 END
      ) AS positive_parties,

      SUM(
        CASE WHEN label_3m = 0
          THEN 1 ELSE 0 END

```

```

        ) AS negative_parties

FROM bu_frd.eda_credit_age_2022_2025

WHERE credit_age_months IS NOT NULL

GROUP BY
    observation_date,
    credit_age_months
)

SELECT
    observation_date,
    credit_age_months,

    positive_parties,
    negative_parties,

    CASE
        WHEN positive_parties > 0
        THEN negative_parties * 1.0 /
            positive_parties
    END AS negatives_per_positive,

    CASE
        WHEN positive_parties > 0
        AND negative_parties = 0
        THEN 1 ELSE 0
    END AS unsupported_positive_stratum

FROM x

WHERE positive_parties > 0

ORDER BY
    observation_date,
    credit_age_months
"
) %>%
collect()

```

credit_age_exact_match_validation

Esta tabla puede ser grande, así que la validación ejecutiva será el siguiente query.

9. Resumen de exact-match support

```
credit_age_exact_match_summary <- sdf_sql(
  sc,
  "
  WITH x AS (

    SELECT
      observation_date,
      credit_age_months,

      SUM(
        CASE WHEN label_3m = 1
          THEN 1 ELSE 0 END
      ) AS positive_parties,

      SUM(
        CASE WHEN label_3m = 0
          THEN 1 ELSE 0 END
      ) AS negative_parties

    FROM bu_frd.eda_credit_age_2022_2025

    WHERE credit_age_months IS NOT NULL

    GROUP BY
      observation_date,
      credit_age_months
  )

  SELECT
    COUNT(
      CASE WHEN positive_parties > 0
        THEN 1 END
    ) AS positive_credit_age_strata,

    SUM(
      CASE
        WHEN positive_parties > 0
          AND negative_parties > 0
        THEN 1 ELSE 0
      END
    ) AS supported_strata,

    SUM(
      CASE
        WHEN positive_parties > 0
```

```

        AND negative_parties = 0
        THEN 1 ELSE 0
    END
) AS unsupported_strata,

SUM(
    CASE
        WHEN positive_parties > 0
        THEN positive_parties ELSE 0
    END
) AS positive_parties,

SUM(
    CASE
        WHEN positive_parties > 0
        AND negative_parties > 0
        THEN positive_parties ELSE 0
    END
) AS positives_with_negative_support,

SUM(
    CASE
        WHEN positive_parties > 0
        AND negative_parties = 0
        THEN positive_parties ELSE 0
    END
) AS positives_without_negative_support

FROM x
"
) %>%
collect()

```

credit_age_exact_match_summary

Esta tabla me interesa muchísimo.

Idealmente queremos:

positives_without_negative_support = 0

o muy cerca de cero.

La métrica conceptual sería:

$$SupportRate = \frac{PositivesWithNegativeSupport}{AllPositivesWithCreditAge}$$

10. Ratio 1:10 por Credit Age

Como nuestra muestra fue diseñada aproximadamente:

1: 10

también podemos revisar si esa relación se mantiene razonablemente a través de credit age:

```
credit_age_ratio_distribution <- sdf_sql(  
  sc,  
  "  
  SELECT  
    credit_age_months,  
  
    SUM(  
      CASE WHEN label_3m = 1  
        THEN 1 ELSE 0 END  
    ) AS positives,  
  
    SUM(  
      CASE WHEN label_3m = 0  
        THEN 1 ELSE 0 END  
    ) AS negatives,  
  
    CASE  
      WHEN SUM(  
        CASE WHEN label_3m = 1  
          THEN 1 ELSE 0 END  
      ) > 0  
  
      THEN  
        SUM(  
          CASE WHEN label_3m = 0  
            THEN 1 ELSE 0 END  
        ) * 1.0  
        /  
        SUM(  
          CASE WHEN label_3m = 1  
            THEN 1 ELSE 0 END  
        )  
    END AS negatives_per_positive  
  
  FROM bu_frd.eda_credit_age_2022_2025  
  
  WHERE credit_age_months IS NOT NULL
```

```

GROUP BY credit_age_months

HAVING SUM(
  CASE WHEN label_3m = 1
    THEN 1 ELSE 0 END
) > 0

ORDER BY credit_age_months
"
) %>%
collect()

```

credit_age_ratio_distribution

No necesitamos que cada mes sea exactamente:

10.000

porque el control sampling original no se hizo necesariamente por credit age.

Pero no queremos regiones completas de credit age en las que haya positives sin comparadores negativos.

Qué resultados quiero revisar primero

Para no generar demasiados resultados, pásame estos seis:

1. `credit_age_integrity_qa`
2. `credit_age_coverage`
3. `credit_age_quality`
4. `credit_age_summary_by_label`
5. `credit_age_smd`
6. `credit_age_exact_match_summary`

El sexto es el más importante para responder textualmente al requerimiento:

“if victims at month 24 of history, then also non victims at that same age.”

Después vemos `credit_age_band_balance` para la gráfica del EDA.

Una precisión metodológica que pondremos en el Word: llamaremos a esto **observable credit-file age**, no customer tenure. Así mantenemos separados los tres conceptos:

```
\[ \boxed{ \begin{array}{ll} 12M \backslash \text{lookback} \ \&= \ \text{behavioral history used for features} \\ \text{CreditAge} \ \&= \ \text{time since earliest observable credit reporting} \\ \text{CustomerTenure} \ \&= \ \text{not reliably available from current sources} \end{array} } \]
```

Eso deja resuelto el potencial conflicto entre las instrucciones y la limitación que ya detectamos con `data_date`.